KiteLang

A Basant-Inspired Programming Language for Algorithms

Compiler Project Proposal

Course: Compiler Construction · Instructor: Dr. Faisal Aslam

Mohammad Ibrahim Salman	23L-0723
Muzammil Ahmed Bhatti	23L-0688
Muhammad Ali Mahoon	23L-0531

1. Purpose

KiteLang is a small programming language for learning and writing algorithms such as searching and sorting. It takes inspiration from Basant, the traditional kite-flying festival, and uses a few familiar kite-flying terms as programming keywords. For example, `pecha` starts a condition, `dheel` repeats instructions, and `dor` moves through a range of values. This gives the language a recognizable identity without making programs difficult to understand.

KiteLang keeps useful parts of C-style programming, including typed variables, arrays, functions, and braces. It avoids advanced features such as pointers and manual memory management. The goal is to make algorithm programs simple to read, practical to compile, and clearly different in syntax from an ordinary C program.

2. Key Features

Feature	What KiteLang provides
Data types	`int`, `float`, `bool`, and `void`.
Variables	Typed declarations such as `int x = 5;`.
Arrays	Fixed-size, one-dimensional arrays such as `int a[6];`.
Functions	Parameters, return values, recursion, and array parameters.
Conditions	`pecha (condition)` and optional `warna` (if/else).
Loops	`dheel (condition)` (while), ordinary `for`, and `dor (i : start -> end)` (range loop).
Swap	`palta(a[i], a[j]);` exchanges two assignable values.
Output	Built-in `print(value);` with no header files.
Comments	Single-line comments beginning with `//`.

3. What Makes KiteLang Different

The Basant words are not only decoration: `dor` is a dedicated inclusive range loop, and `palta` is a built-in swap statement. Both are especially useful when writing sorting algorithms. A `dor` loop runs from its starting value to its ending value, including both ends, with a step of one; its bounds are evaluated when the loop begins. The first version supports increasing ranges only. `palta` swaps two variables or array elements of the same type. These features can be lowered into ordinary control flow and assignments during compilation, so they do not need a complex runtime.

```
pecha (a > b) { print(a); } warna { print(b); }
dor (i : 0 -> 5) { print(data[i]); }
palta(data[0], data[1]);
```

4. How the Compiler Works

KiteLang uses ahead-of-time (AOT) compilation. It reads a `.kite` source file and creates a native executable. The three required technologies are Flex, Bison, and LLVM.

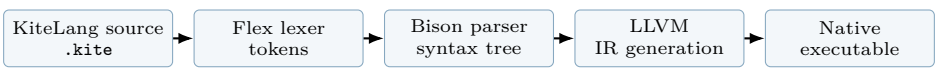


Figure 1. KiteLang compilation pipeline.

1. **Flex:** Recognizes names, numbers, operators, and special words such as `pecha`, `dor`, and `palta`.
2. **Bison:** Checks the grammar and builds a syntax tree for conditions, loops, functions, and statements.
3. **LLVM:** Checks basic types and converts the syntax tree into LLVM intermediate representation (IR). The new statements become ordinary branches, loops, and assignments.
4. **Native executable:** LLVM produces machine code that can be linked and run.

5. Project Scope

Included	Not included
Basic typed variables and fixed-size arrays	Pointers and dynamic memory
`pecha`/`warna`, `dheel`, `for`, `dor`	Classes, structs, and multidimensional arrays
Functions, recursion, and `palta`	Multiple source files and external libraries
Basic type checking and error messages	Preprocessor and advanced optimizations
LLVM IR and AOT native compilation	JIT compilation

6. Example Program: Insertion Sort

This program uses an array, variables, a function, loops, and a `pecha`/`warna` condition. The `palta` statement exchanges adjacent elements until the array is sorted. The `dor` loop prints the final result.

```
// Sort numbers in ascending order
void insertionSort(int a[], int n) {
    dor (i : 1 -> n - 1) {
        int j = i;
        dheel (j > 0) {
            pecha (a[j] < a[j - 1]) {
                palta(a[j], a[j - 1]);
                j = j - 1;
            } warna {
                j = 0;
            }
        }
    }
}

int main() {
    int data[6] = {9, 4, 7, 2, 8, 1};
    insertionSort(data, 6);
    dor (i : 0 -> 5) {
        print(data[i]);
    }
    return 0;
}
```

Expected output: `1 2 4 7 8 9` (one number per line).

Success criterion: The project is complete when the KiteLang compiler accepts this program, generates LLVM IR, creates a native executable, and prints the sorted numbers correctly. The language will also be designed to express merge sort using arrays, functions, recursion, and its control-flow statements.